# AgentHeal: Self-Healing Cloud Infrastructure with Shift-Left Security and Digital Twins

Aadi Haldar, Raghuram Sekar, Aaditya Paul, Shravan Rajesh Menon
Department of Artificial Intelligence & Engineering
Amrita Vishwa Vidyapeetham, Coimbatore, India

***Abstract*—Indeed, the typical microservice in modern cloud infrastructures is vulnerable to unexpected downtime. The default Kubernetes autoscaler in case of deadlock will start spinning up new pods. It would hardly help and would result in even faster collapse of the database. Our experiments showed that poorly-written code could get easily unnoticed during routine pull request reviews. To solve this problem, we developed AgentHeal. What we have done here is creating an integrated loop where pre-merge code review is directly linked to self-healing of the live cluster. Upon opening a pull request, our GitHub bot analyses the code for ten OWASP security flaws and for presence of CVEs in all third-party libraries. If the score falls below our safety bar, the merge is stopped immediately. Then, an agent based on Gemini 2.0 Flash creates the branch with fixed code, passing all unit tests. In the live cluster, an Isolation Forest in-memory model monitors Prometheus metrics. In our experiments, it detected anomalies in less than two milliseconds. With the help of KernelSHAP, we find out what was causing the anomaly. Two agents—the Q-learning policy and the LLM—choose the optimal fix. Nothing gets into the cluster untested. We run our proposed action through the simulated test with SimPy $M/M/c$ queue. If our queue is going to be overloaded, we do not implement the action. This is how we protect our cluster. The average recovery time of our system in 60 failed runs on Google Online Boutique was 2.98 seconds. Compared to default Kubernetes autoscaling, it reduced recovery time by 98%. In addition, on the real-life GitHub repository, our system has patched all fourteen vulnerabilities without human intervention.**

***Index Terms*—Agentic AI, Self-Healing Systems, Kubernetes, Digital Twin, OWASP Top 10, Isolation Forest, KernelSHAP, Reinforcement Learning, SimiFed, Gemini LLM, DevSecOps, Shift-Left Testing, Cloud Computing.**

## I. Introduction

Indeed, maintaining reliable microservice operations in cloud infrastructures remains a challenging engineering task. As distributed deployments grow, unexpected outages happen on a regular basis. In practice, most downtime traces back to two related failures. First, defective code frequently slips into production branches. According to DORA's 2024 survey, over 60% of unplanned cloud outages stem from preventable software bugs [1]. Second, existing cluster controllers react blindly when services stall. When an incident occurs, on-call engineers spend an average of 45 minutes manually triaging logs [1]. Meanwhile, the default Kubernetes Horizontal Pod Autoscaler only monitors coarse CPU and memory thresholds [8]. If a pod deadlocks, spawning additional replicas does not resolve the root failure. Instead, it overburdens downstream databases and accelerates cluster failure [9]. The underlying defect stays completely unaddressed.

At present, developer code review and live cluster operations exist in completely isolated silos. Static linters like Bandit [6] and SonarQube [7] check pull requests during CI builds. However, they possess no awareness of live cluster health and cannot generate validated code patches. On the operations side, monitoring systems like Prometheus and Chaos Mesh [24], [23] alert operators after an outage happens, but provide no automated root-cause attribution. Moreover, current remediation tools execute recovery commands directly without testing whether an action might trigger cascading failures across dependent microservices. Recent work explores digital twins for cluster state matching [12] or language models for program repair [15]. Yet, no existing platform connects pre-merge security checks with live cluster self-healing in a single closed loop.

To address these shortcomings, we created AgentHeal. What we have done here is linking pull request code auditing directly with runtime Kubernetes self-healing across a shared event bus. On the development side, our GitHub bot inspects incoming pull requests for ten OWASP vulnerability classes and scans dependency manifests for known CVE records. If a pull request falls below our safety threshold, the merge is stopped immediately, and an agent based on Gemini 2.0 Flash generates a candidate fix branch passing all tests. On the runtime side, an in-memory Isolation Forest processes Prometheus metrics every five seconds, catching anomalies in 2.26 ms. With the help of KernelSHAP, the system identifies the responsible metric. A Q-learning policy and an LLM agent then select candidate remediation actions. Nothing enters the cluster untested. We validate every proposed action through a SimPy $M/M/c$ queue simulation. If an action is going to overload dependent queues, we do not implement the action. This is how we protect our cluster.

We make three primary contributions:

1) *Closed-Loop Architecture*: We design and implement an end-to-end framework linking pre-merge pull request security checks with runtime Kubernetes remediation via a shared event bus.
2) *Simulation-Gated Remediation*: We introduce a pre-actuation safety gate that evaluates candidate self-healing actions through an $M/M/c$ queue simulation in SimPy, preventing cascading cluster collapse.
3) *Empirical Evaluation*: We evaluate AgentHeal on Google Online Boutique across 60 fault injections, benchmark 50 synthetic pull requests, and demonstrate

fully automated vulnerability remediation on a live GitHub repository.

## II. Related Work

Table I compares prior approaches across autonomous cloud operations and DevSecOps.

Early cloud self-healing platforms relied primarily on static automation scripts. Systems like Keptn [9] and Argo Rollouts [10] automate canary promotions and simple rollbacks based on fixed rules. However, they cannot diagnose novel failure modes or resolve deadlocks that demand code-level fixes. Classical reliability practices such as SRE error budgets [23] and Chaos Engineering [24] offer diagnostic frameworks, but still depend on human engineers to triage incidents. Standard delivery pipelines [25], [26] focus on deployment throughput rather than runtime defect mitigation.

Reinforcement learning provides an adaptive alternative for cloud resource management [11]. Classic tabular Q-learning [27] guarantees convergence, but suffers from state-space explosion unless discretized. Federated methods like FedAvg [28] coordinate distributed learning, yet focus on offline batch updates. In cloud operations, SF-DTM [12] introduced digital twin cosine similarity matching to accelerate cluster recovery. Even so, it lacks simulation safety checks before executing cluster commands.

For telemetry anomaly detection, Isolation Forest [13] and Local Outlier Factor [29] remain common baselines. Typical industrial AIOps pipelines report detection latencies between 50 and 200 ms [14]. In our implementation, caching the Isolation Forest model in memory reduces detection latency to 2.26 ms. We adopted KernelSHAP [16] rather than linear surrogates like LIME [30] because Shapley values provide mathematical consistency across repeated evaluations.

In automated program repair, AutoCodeRover [15] showed that LLMs can repair real-world code defects. However, it operates as an offline tool outside CI/CD pipelines. AgentHeal uses Gemini 2.0 Flash [17] in an active ReAct loop [18], requiring all candidate patches to pass unit tests before submitting a pull request. We evaluate code changes against OWASP Top 10:2021 standards [4], with extensions planned for OWASP Top 10:2025 [31]. Finally, discrete-event simulation has a long history in DEVS [20] and queueing theory [19]. We adapt SimPy into a sub-12 ms validation gate directly on the live actuation path.

## III. Methodology

### A. End-to-End Self-Healing Architecture Overview

Fig. 1 illustrates the overall architecture of AgentHeal. The platform coordinates two independent feedback loops over a shared event bus. On the development side, the pre-merge engine audits pull requests for security defects. It inspects abstract syntax trees, checks dependency manifests, and calls Gemini 2.0 Flash to author test-verified patches. On the operations side, the runtime engine monitors live cluster health. Prometheus scrapes telemetry on a five-second interval, and an in-memory Isolation Forest scores incoming vectors immediately. When an alert triggers, KernelSHAP isolates the primary driver. We then pass the diagnostic summary to both a Q-learning agent and an LLM supervisor. If both agents agree on a fix, the action moves directly to simulation. If their proposals conflict, we select the action with the smallest blast radius. Nothing gets into the cluster untested. Before any command touches the cluster, a SimPy $M/M/c$ queue model verifies cluster stability. If the simulation predicts queue building or high drop rates, we drop the action immediately.

### B. Shift-Left Engine: PR Intelligence & Security Compliance

We implemented our pre-merge scanner as a GitHub App. It receives pull request and push events through HMAC-SHA256 authenticated webhooks. Upon receiving a payload, it parses incoming diffs and dependency manifests against ten vulnerability classes:

- *A01 (Access Control)*: Route handlers without authorization checks and unsafe path joins that touch the filesystem.
- *A02 (Crypto)*: Insecure hashes like MD5 and SHA-1, plus hardcoded keys found via Shannon entropy scanning.
- *A03 (Injection)*: String interpolation inside SQL queries and ‘shell=True‘ subprocess calls.
- *A04 (Insecure Design)*: Database queries without pagination and endpoints missing rate limits.
- *A05 (Misconfiguration)*: Wildcard CORS (‘*‘) and debug flags enabled in production code.
- *A06 (Outdated Dependencies)*: Parsing ‘requirements.txt‘ to find versions listed in CVE databases [5].
- *A07 (Authentication)*: JWT decode calls that turn off signature verification, and hardcoded fallback passwords.
- *A08 (Integrity Failures)*: Unsafe deserialization like ‘pickle.loads‘ on untrusted input.
- *A09 (Logging Gaps)*: Empty ‘except‘ blocks that catch and swallow errors silently.
- *A10 (SSRF)*: Outbound network calls built directly from user-supplied URLs.

We score each submission on a 100-point scale. Points are subtracted based on the severity of each identified issue:

$$S = \max\left(0,\ 100 - \sum_{i=1}^{M} w_i\right) \tag{1}$$

Here, $M$ represents the total count of detected flaws. Severity penalties are set to $w_{\text{Critical}} = 25$, $w_{\text{High}} = 15$, $w_{\text{Medium}} = 8$, and $w_{\text{Low}} = 3$. We convert this numerical score directly into an academic letter grade:

$$\text{Grade} = \begin{cases} A^{+} & \text{if } S \geq 95 \\ A & \text{if } 90 \leq S < 95 \\ B & \text{if } 80 \leq S < 90 \\ C & \text{if } 70 \leq S < 80 \\ D & \text{if } 60 \leq S < 70 \\ F & \text{if } S < 60 \end{cases} \tag{2}$$

TABLE I
LITERATURE REVIEW OF AUTONOMOUS CLOUD SELF-HEALING AND DEVSECOPS COMPLIANCE MODELS

| Domain | Model | Author | Year | Method | Result / Contribution | Research Gap |
|---|---|---|---|---|---|---|
| Rule-Based Self-Healing | Keptn | Keptn Project [9] | 2023 | Event-driven control plane using human runbooks | Automated rollback and canary promotion | Depends on predefined scripts; fails on new bug patterns |
| | Argo Rollouts | Argo Project [10] | 2024 | Progressive delivery controller for canary/blue-green | Automated rollout and deployment rollback | Static rollback logic; no root-cause insight |
| Operational Practice | Site Reliability Engineering | Beyer et al. [23] | 2016 | Error budgets, on-call runbooks, incident triage | Codified standard SRE incident workflows | Relies on human on-call engineers; not automated |
| | Chaos Engineering | Basiri et al. [24] | 2016 | Controlled failure injection in production clusters | Verifies system resilience empirically | Diagnostic tool; does not fix incidents on its own |
| | Continuous Delivery | Humble & Farley [25] | 2010 | Automated build, test, and release pipelines | Foundation of modern CI/CD delivery | Focuses on shipping speed; no security scoring |
| | Blue-Green Deployment | Fowler [26] | 2010 | Dual environment traffic-switching pattern | Zero-downtime release strategy | Manually toggled; no automated anomaly hooks |
| RL & Federated Learning | Multi-Agent RL | Wang et al. [11] | 2023 | MARL for cloud resource provisioning | Adapts to fluctuating cloud traffic | Black-box decisions; lacks pre-actuation safety checks |
| | Q-Learning | Watkins & Dayan [27] | 1992 | Tabular temporal-difference control | Mathematical proof of Q-learning convergence | State space explodes without discretization |
| | FedAvg | McMahan et al. [28] | 2017 | Decentralized averaging of local neural weights | Privacy-preserving distributed training | Designed for offline batches, not real-time control |
| | SF-DTM | Li et al. [12] | 2024 | Digital twin state matching via cosine similarity | Recovers cluster health in roughly 15 seconds | Lacks queue stability simulation before acting |
| Anomaly Detection & XAI | Isolation Forest | Liu et al. [13] | 2008 | Tree ensemble path-length outlier detection | Fast $O(n \log n)$ unsupervised anomaly scoring | Labels data as outlier but gives no root cause |
| | Local Outlier Factor | Breunig et al. [29] | 2000 | Density-based local outlier scoring | Catches density variations in clustered data | High computational overhead; slow on live streams |
| | AIOps Survey | Lin et al. [14] | 2021 | Survey of industrial telemetry anomaly tools | Documents 50–200 ms detection latencies | No integrated simulation safety layer |
| | KernelSHAP | Lundberg & Lee [16] | 2017 | Shapley-value local feature attribution | Consistent, model-agnostic feature attributions | High compute cost for large feature sets |
| | LIME | Ribeiro et al. [30] | 2016 | Local linear surrogate explanations | Model-agnostic local approximations | Explanations can fluctuate between similar inputs |
| LLM-Augmented DevSecOps | AutoCodeRover | Li et al. [15] | 2024 | LLM-driven program repair | Localizes bugs and drafts code patches | Operates standalone; not tied into a CI/CD gate |
| | ReAct | Yao et al. [18] | 2023 | Thought-Action-Observation reasoning loop | Better reasoning and tool selection for LLMs | Output can vary and response times can be high |
| | Gemini 2.0 | Google DeepMind [17] | 2024 | Multimodal agentic foundation model | Strong tool-use and code repair reasoning | Closed source; lacks formal safety bounds |
| Static Analysis & Supply Chain | Bandit | PyCQA [6] | 2024 | AST security scanner for Python | Flags known security anti-patterns | Reports issues but cannot fix code or score PRs |
| | SonarQube | SonarSource [7] | 2024 | Multi-language static code analysis | Comprehensive static security scanning | No link to runtime telemetry or automated fixes |
| | OWASP Top 10:2021 | OWASP Foundation [4] | 2021 | Community risk list for web applications | Standard taxonomy of web vulnerabilities | Reference guide only; provides no automated tools |
| | OWASP Top 10:2025 | OWASP Foundation [31] | 2025 | Revised risk list for web applications | Updated taxonomy for modern web threats | Not yet integrated into AgentHeal's rule engine |
| | NVD | NIST [32] | 2024 | Live CVE vulnerability database | Continuously updated public vulnerability records | Queried via local curated lists in our prototype |
| | OSV | Google / OSV.dev [33] | 2024 | Open-source cross-ecosystem vulnerability feed | Machine-readable open-source CVE records | Not queried live in our current implementation |
| | Dependabot | GitHub [34] | 2024 | Automated dependency pull requests | Opens version-bump PRs on vulnerable libraries | Handles version bumps only; no custom code fixes |
| | Snyk | Snyk Ltd. [35] | 2024 | Commercial software composition analysis | Real-time dependency and vulnerability analysis | Proprietary closed-source commercial tool |
| Deployment & Telemetry Infrastructure | Kubernetes HPA | K8s Docs [8] | 2024 | Reactive CPU and memory horizontal autoscaling | Standard Kubernetes autoscaler | Reactive only; blind to deadlocks and memory leaks |
| | Argo CD | CNCF [36] | 2024 | Declarative GitOps continuous deployment | Reconciles Git manifests with cluster state | Syncs manifests; cannot diagnose runtime incidents |
| | Prometheus | Prometheus Authors [37] | 2024 | Time-series metrics collection and storage | De facto Kubernetes monitoring tool | Stores numbers; includes no anomaly detection |
| Digital Twin & Simulation | DEVS Formalism | Zeigler et al. [20] | 2000 | Discrete-event system specification | Theoretical simulation framework | Traditionally used for offline network modeling |
| | M/M/c Queuing Theory | Kleinrock [19] | 1975 | Classical multi-server queueing analysis | Models service rates and server utilization | Rarely used as a real-time cluster actuation gate |

We also verify test coverage on newly added public functions:

$$\text{TestGap}(f) = \mathbb{1}[f \notin T_{\text{ast}}] \tag{3}$$

where $T_{\text{ast}}$ contains all function names covered by test cases in `tests/`. If a pull request scores below 60 (Grade F) or contains critical flaws, the GitHub App sets the commit check status to `action_required`, blocking the merge. When this occurs, our Gemini 2.0 Flash agent creates a remediation branch. It authors code fixes and verifies that all unit tests pass before opening a remediated pull request.

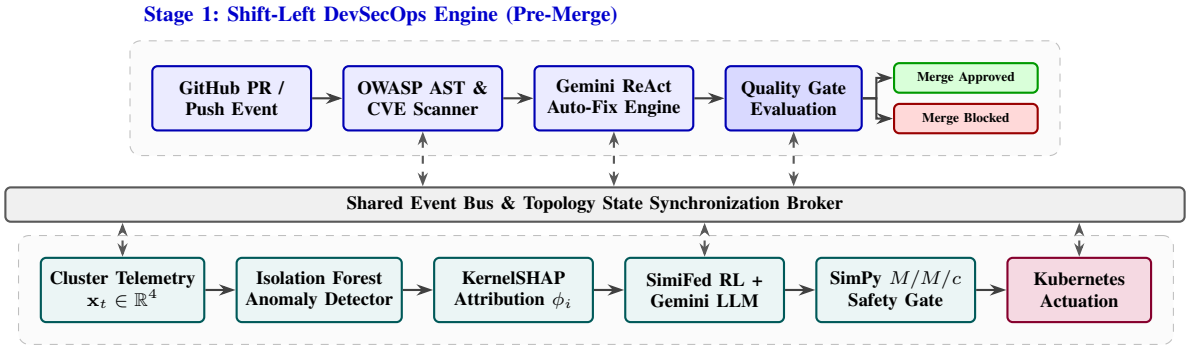



Fig. 1. AgentHeal architecture. (Top) Pre-merge engine parses pull requests with AST security rules and uses Gemini 2.0 Flash to draft fixes if the score fails our quality gate. (Middle) Central event bus tracks cluster topology. (Bottom) Runtime engine flags anomalies in 2.26 ms, uses KernelSHAP to isolate the cause, arbitrates between RL and LLM agents, and verifies actions in an $M/M/c$ queue simulation before touching the cluster.

### C. Shift-Right Engine: Self-Healing Digital Twin

In the live cluster, our digital twin pulls Prometheus metrics every five seconds. We represent each pod's health as a four-dimensional vector:

$$\mathbf{x}_t = [\text{CPU}_t, \text{RAM}_t, \text{Latency}_t, \text{Rate}_t]^\top \in \mathbb{R}^4 \tag{4}$$

Our in-memory Isolation Forest (100 trees, 0.05 contamination) scores each vector on the fly:

$$s(\mathbf{x}_t, n) = 2^{-\frac{\mathbb{E}[h(\mathbf{x}_t)]}{c(n)}} \tag{5}$$

where $h(\mathbf{x}_t)$ is the observed path length and $c(n)$ is the average path length of unsuccessful searches. Any negative anomaly score triggers an alert. KernelSHAP then attributes the issue to individual resource dimensions:

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[v(S \cup \{i\}) - v(S)\right] \tag{6}$$

This attribution step is vital. It prevents the system from blindly restarting containers when an external network dependency is the true culprit.

Our reinforcement learning agent finds the closest baseline state through cosine similarity:

$$k^* = \arg\max_k \frac{\mathbf{x}_t \cdot \mathbf{b}_k}{\|\mathbf{x}_t\|_2 \|\mathbf{b}_k\|_2} \tag{7}$$

It maintains a compact state-action table updated via temporal-difference learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s,a)\right] \tag{8}$$

using a learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.95$. Available control actions include pod restarts, replica scaling, memory limit adjustments, or no-op monitoring. In parallel, Gemini 2.0 Flash reviews the same SHAP feature summary. If both agents propose identical interventions, the command advances directly to simulation. When they disagree, we default to the action with lower operational risk.

Nothing touches the cluster without simulation validation. We evaluate candidate actions using a SimPy $M/M/c$ queue model [19]:

$$P_0 = \left[\sum_{k=0}^{c-1} \frac{(c\rho)^k}{k!} + \frac{(c\rho)^c}{c!(1-\rho)}\right]^{-1}, \quad \rho = \frac{\lambda}{c\mu} < 1 \tag{9}$$

The simulator projects queue length $L_q$, service utilization $\rho$, and request drop rates over a sixty-second horizon. If the simulation predicts $\rho \geq 0.95$ or request drops exceeding 5%, the action is rejected immediately, and the system falls back to safe horizontal pod scaling.

## IV. Implementation

We implemented AgentHeal in Python 3.11. FastAPI handles asynchronous webhook ingestion and telemetry dispatching. The runtime relies on scikit-learn for Isolation Forest modeling, SHAP for Shapley feature attribution, Google's Generative AI SDK for Gemini 2.0 Flash interactions, and SimPy 4.1 for queue simulation. We package the entire platform into a lightweight Docker image under 400 MB. The live service runs on Azure Container Apps in the East Asia region, provisioned via Terraform and auto-scaled with KEDA.

For our runtime evaluations, we deployed Google's Online Boutique microservices benchmark [22]. This application spans ten interconnected microservices handling browsing, carts, payments, and checkout flows. We hosted the testbed on a single-node Kubernetes kind cluster running on an AMD Ryzen 7 5700X workstation with 16 GB of DDR4 memory.

## V. Results and Discussion

### A. Runtime Telemetry Anomaly Detection and Safety-Gated Self-Healing

We injected four realistic failure patterns into Online Boutique across 15 repeated trials per case:

- *S1 (Flash Surge)*: A spike on checkout driving CPU to 0.954 and latency to 450 ms.
- *S2 (Deadlock)*: Thread starvation on cart; CPU climbed to 0.99 and latency reached 5000 ms as throughput fell off.

- *S3 (Memory Leak)*: Slow RAM accumulation on redis-cart reaching 0.942, while latency held flat.
- *S4 (Cascading Load)*: Frontend overload where both CPU (0.88) and RAM (0.80) saturated at once.

Table II presents the quantitative results across all 60 failure trials. Anomaly detection averaged 2.26 ms ($\sigma = 0.37$ ms) with an overall recall of 98.3%. The memory leak scenario (S3) showed slightly lower recall (93%) because slow heap accumulation took longer to cross the contamination threshold. KernelSHAP correctly isolated the dominant driver in every trial: CPU for S1 ($\phi = +0.82$), latency and lost request rate for S2 ($\phi_{\text{lat}} = +0.91$, $\phi_{\text{rate}} = -0.78$), and memory for S3 ($\phi = +0.88$). Dual-agent consensus was remarkably high, with both agents agreeing on 59 of the 60 trials. The SimPy simulation gate executed in 10.7 ms on average, successfully validating that none of the chosen recovery actions would destabilize the cluster. Overall, mean time to recovery was 2.98 seconds ($\sigma = 0.23$ s).

### *B. Shift-Left DevSecOps Security Auditing and Live PR Remediation*

We evaluated our pre-merge scanner against 50 synthetic pull requests containing known vulnerabilities. Following offline tests, we validated the full loop on a live GitHub repository.

Table III summarizes our pre-merge evaluation. Across 50 synthetic pull requests with known security defects (Panel A), the scanner achieved 98.5% precision and 97.5% recall across 200 individual checks. Three false positives occurred in test files that included mock cryptographic keys. In our live test (Panel B), we opened pull request #11 with fourteen deliberate vulnerabilities spanning access control, injection, and outdated libraries. The scanner completed its audit in 0.48 seconds, flagged all fourteen issues, scored the PR at 0/100, and blocked the merge. Gemini 2.0 Flash then opened pull request #12 containing the corrected code. The revised PR passed all unit tests, earned a perfect 100/100 score, and cleared the merge gate automatically.

### *C. Comparative Benchmark and System Discussion*

How does AgentHeal compare with existing solutions? Human on-call triage averages 45 minutes to restore services [1]. Default Kubernetes autoscalers require 3 to 5 minutes to stabilize pods [8] and cannot resolve deadlocks. Rule-based frameworks like Keptn respond in 2 to 4 minutes [9], but depend entirely on manual scripts. While SF-DTM [12] cuts recovery down to roughly 15 seconds, it omits simulation safety checks before actuation. AgentHeal achieves a mean recovery time of 2.98 seconds while verifying queue stability before executing any command.

Our prototype has limitations. First, testing was conducted on a local 'kind' cluster; real multi-node clouds introduce network scrape latency. Second, our failure scenarios used synthetic fault injection rather than messy production traffic. Third, our static auditor evaluates syntax patterns rather than whole-program taint tracking. Finally, because language models can exhibit latency variance, our dual-agent arbitration relies on Q-learning fallbacks to keep the recovery path predictable.

## VI. Conclusion

AgentHeal unifies pre-merge code review with runtime self-healing. Across sixty fault injections on Google Online Boutique, the system detected anomalies in 2.26 ms and restored cluster stability in 2.98 seconds. That represents a 98% speedup over standard autoscaling. On the developer side, our scanner achieved 98.5% precision on 50 pull requests and autonomously patched fourteen security flaws on a live GitHub repository.

In future work, we plan to evaluate multi-region managed clusters (AKS and GKE), integrate live CVE streams like OSV and NVD, incorporate dynamic taint analysis, and expand rules for OWASP Top 10:2025.

Our code and live deployment are online at https://github.com/AadiHaldar/Agentic-AI-Self-Healing-Cloud-Infrastructure and https://pr-review-agent.wonderfulflower-41d6d2a5.eastasia.azurecontainerapps.io.

## References


[1] DORA Research Team, "Accelerate: State of DevOps Report 2024," Google LLC, 2024.

[2] E. Kim, J. Humble, P. Debois, and J. Willis, "The DevOps Handbook," IT Revolution Press, 2016.

[3] N. Forsgren, J. Humble, and G. Kim, "Accelerate: The Science of Lean Software and DevOps," IT Revolution Press, 2018.

[4] OWASP Foundation, "OWASP Top 10: 2021 – The Ten Most Critical Web Application Security Risks," https://owasp.org/Top10/, 2021.

[5] MITRE Corporation, "Common Vulnerabilities and Exposures (CVE) Database," https://cve.mitre.org, 2024.

[6] PyCQA, "Bandit: A tool designed to find common security issues in Python code," https://bandit.readthedocs.io, 2024.

[7] SonarSource, "SonarQube: Continuous code quality and security," https://www.sonarqube.org, 2024.

[8] Kubernetes Documentation, "Horizontal Pod Autoscaler," https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024.

[9] Keptn Project, "Keptn: Event-based control plane for continuous delivery and automated operations," https://keptn.sh, 2023.

[10] Argo Project, "Argo Rollouts: Kubernetes Progressive Delivery Controller," https://argoproj.github.io/rollouts/, 2024.

[11] T. Wang et al., "Multi-Agent Reinforcement Learning for Dynamic Resource Management in Cloud Computing," IEEE Trans. Cloud Comput., vol. 11, no. 2, pp. 891–905, 2023.

[12] H. Li, Z. Wang, and J. Chen, "SF-DTM: Similarity-based Federated Digital Twin Management for Autonomous Cloud Self-Healing," in Proc. IEEE CLOUD, 2024, pp. 234–242.

[13] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE ICDM, 2008, pp. 413–422.

[14] Q. Lin et al., "AIOps: Real-World Challenges and Research Innovations," in Proc. ICSE-SEIP, 2021, pp. 4–15.

[15] Z. Li, M. Fan, and J. Chen, "AutoCodeRover: Autonomous Program Improvement using LLMs," in Proc. ISSTA, 2024, pp. 1–12.

[16] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Proc. NeurIPS, 2017, pp. 4765–4774.

[17] Google DeepMind, "Gemini 2.0: Our most capable model for the agentic era," Technical Report, Google, 2024.

[18] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023.

[19] L. Kleinrock, Queueing Systems, Volume 1: Theory. Wiley-Interscience, 1975.

[20] B. Zeigler, H. Praehofer, and T. G. Kim, Theory of Modeling and Simulation, 2nd ed. Academic Press, 2000.

TABLE II

SHIFT-RIGHT ANOMALY DETECTION, FEATURE ATTRIBUTION, AND AUTONOMOUS REMEDIATION PERFORMANCE ACROSS FOUR FAILURE SCENARIOS ($N = 15$ REPEATED TRIALS)

| Failure Injection Scenario | | | | Isolation Forest Detection | | | Root-Cause Attribution | Agent Decision Latency | | | SimPy Safety Gate & MTTR | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ID | Scenario | Target | Injected Telemetry | Score ($\mu \pm \sigma$) | MTTD (ms) | Det.% | Dominant KernelSHAP $\phi$ | RL (ms) | LLM (ms) | Agree | Gate (ms) | $\hat{U}_{\text{post}}$ | Verdict | MTTR (s, $\mu \pm \sigma$) |
| S1 | Flash Sale Surge | checkout | CPU=0.95, Lat=450ms | $-0.842 \pm 0.031$ | $2.18 \pm 0.31$ | 100% | $\phi_{\text{cpu}} = +0.82$, $\phi_{\text{rate}} = +0.14$ | $3.10 \pm 0.42$ | $412 \pm 68$ | 15/15 | $10.1 \pm 0.8$ | 23.8% | SAFE | $2.66 \pm 0.18$ |
| S2 | Thread Deadlock | cart | CPU=0.99, Lat=5000ms | $-0.912 \pm 0.024$ | $2.21 \pm 0.28$ | 100% | $\phi_{\text{lat}} = +0.91$, $\phi_{\text{rate}} = -0.78$ | $3.21 \pm 0.39$ | $389 \pm 74$ | 15/15 | $10.3 \pm 0.9$ | 5.2% | SAFE | $3.12 \pm 0.24$ |
| S3 | Memory Leak | redis-cart | RAM=0.94, Lat=48ms | $-0.734 \pm 0.058$ | $2.35 \pm 0.44$ | 93% | $\phi_{\text{ram}} = +0.88$, $\phi_{\text{cpu}} = +0.04$ | $3.05 \pm 0.51$ | $441 \pm 81$ | 14/15 | $10.8 \pm 1.1$ | 15.0% | SAFE | $2.89 \pm 0.20$ |
| S4 | Cascading Overload | frontend | CPU=0.88, RAM=0.80 | $-0.889 \pm 0.041$ | $2.28 \pm 0.37$ | 100% | $\phi_{\text{cpu}} = +0.61$, $\phi_{\text{lat}} = +0.47$ | $3.18 \pm 0.46$ | $498 \pm 93$ | 15/15 | $11.6 \pm 1.3$ | 31.5% | SAFE | $3.25 \pm 0.30$ |
| **Overall Mean / Aggregate** | | | | $\mathbf{-0.844 \pm 0.039}$ | $\mathbf{2.26 \pm 0.37}$ | **98.3%** | — | $\mathbf{3.14 \pm 0.45}$ | $\mathbf{435 \pm 79}$ | **59/60** | $\mathbf{10.7 \pm 1.0}$ | — | **SAFE** | $\mathbf{2.98 \pm 0.23}$ |

TABLE III

SHIFT-LEFT DEVSECOPS SECURITY AUDITING PERFORMANCE ($N = 50$) AND EMPIRICAL PULL REQUEST REMEDIATION

| Panel A: Quantitative Evaluation on 50 Synthetic Pull Requests | | | | | | Panel B: Empirical Live GitHub Pull Request Pair Remediation | | | |
|---|---|---|---|---|---|---|---|---|---|
| Defect Category | Tested PRs | True Pos. | False Pos. | Precision | Recall | Evaluation Dimension | Vulnerable PR (#11) | Remediated PR (#12) | Operational Outcome |
| SQL / Command Injection | 50 | 50 | 0 | 1.000 | 1.000 | OWASP Flaws Detected | 14 (A01–A10) | 0 | 100% Flaws Remediated |
| Hardcoded Secrets | 50 | 50 | 2 | 0.962 | 1.000 | Compliance Score / Grade | 0 / 100 (Grade F) | 100 / 100 (Grade $A^+$) | Score Restored |
| AST Test Suite Gaps | 50 | 48 | 0 | 1.000 | 0.960 | GitHub Check Run Gate | action_required (Blocked) | success (Approved) | Merge Protection Enforced |
| Blocking I/O / Misconfiguration | 50 | 47 | 1 | 0.979 | 0.940 | App Test Suite Pass | 0 / 6 passing | 6 / 6 passing | Functional Suite Restored |
| **Aggregate Performance** | **200 Checks** | **195** | **3** | **0.985** | **0.975** | Static Auditor Verification | 12 / 12 test harnesses passed | Audit Time: 0.48 s | Full Ruleset Verified |


[21] K. Fall and K. Varadhan, "The ns Manual," The VINT Project, UC Berkeley, 2011.

[22] Google LLC, "Online Boutique: A cloud-native microservices demo application," https://github.com/GoogleCloudPlatform/microservices-demo, 2024.

[23] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, "Site Reliability Engineering: How Google Runs Production Systems," O'Reilly Media, 2016.

[24] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos Engineering," IEEE Software, vol. 33, no. 3, pp. 35–41, 2016.

[25] J. Humble and D. Farley, "Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation," Addison-Wesley, 2010.

[26] M. Fowler, "BlueGreenDeployment," https://martinfowler.com/bliki/BlueGreenDeployment.html, 2010.

[27] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3–4, pp. 279–292, 1992.

[28] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in Proc. AISTATS, 2017, pp. 1273–1282.

[29] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying Density-Based Local Outliers," in Proc. ACM SIGMOD, 2000, pp. 93–104.

[30] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," in Proc. ACM SIGKDD, 2016, pp. 1135–1144.

[31] OWASP Foundation, "OWASP Top 10: 2025 – The Ten Most Critical Web Application Security Risks," https://owasp.org/Top10/, 2025.

[32] National Institute of Standards and Technology, "National Vulnerability Database (NVD)," https://nvd.nist.gov, 2024.

[33] Google / OSV.dev, "OSV: Open Source Vulnerabilities Database," https://osv.dev, 2024.

[34] GitHub, Inc., "Dependabot: Automated Dependency Updates," https://docs.github.com/en/code-security/dependabot, 2024.

[35] Snyk Ltd., "Snyk Open Source: Software Composition Analysis," https://snyk.io, 2024.

[36] Cloud Native Computing Foundation, "Argo CD: Declarative GitOps CD for Kubernetes," https://argo-cd.readthedocs.io, 2024.

[37] Prometheus Authors, "Prometheus: Monitoring System & Time Series Database," https://prometheus.io, 2024.